3 Building microservices with Spring Boot

This chapter covers

- Understanding how microservices fit into a cloud architecture
- Decomposing a business domain into a set of microservices
- Understanding the perspectives for building microservice apps
- Learning when not to use microservices
- Implementing a microservice

To successfully design and build microservices, you need to approach them as if you're a police detective interviewing witnesses to a crime. Although each witness sees the same event, their interpretation of the crime is shaped by their background, what's important to them (for example, what motivates them), and what environmental pressures were brought to bear at the moment they witnessed the event. Witnesses each have their own perspective (and bias) on what they consider essential.

Like successful police detectives trying to get to the truth, the journey to build a successful microservice architecture involves incorporating the perspectives of multiple individuals within your software development organization. Because it takes more than technical people to deliver an entire application, we believe that the foundation for successful microservice development starts with the perspectives of three critical roles:

- *The architect*—Sees the big picture, decomposes an application into individual microservices, and then understands how the microservices interact to deliver a solution.
- *The software developer*—Writes the code and understands how the language and development frameworks will be used to deliver a microservice.
- *The DevOps engineer*—Determines how the services are deployed and managed throughout production and non-production environments. The watchwords for the DevOps engineer are *consistency* and *repeatability* in every environment.

In this chapter, we will demonstrate how to design and build a set of microservices from the perspective of each of these roles. This chapter will give you the foundation you need to identify potential microservices within your own business application and then

to understand the operational attributes that need to be in place to deploy a microservice. By the end of this chapter, you'll have a service that can be packaged and deployed to the cloud using the skeleton project we created in the chapter 2.

3.1 The architect's story: Designing the microservice architecture

An architect's role in a software project is to provide a working model of the problem that needs to be solved. The architect provides the scaffolding against which developers will build their code so that all the pieces of the application fit together. When building a microservice, a project's architect focuses on three key tasks:

- Decomposing the business problem
- Establishing service granularity
- Defining the service interfaces

3.1.1 Decomposing the business problem

In the face of complexity, most people try to break down the problem on which they're working into manageable chunks. They do this so they don't have to fit all the details of the problem in their heads. They may break the problem down into a few essential parts and then look for the relationships that exist among these parts.

In a microservices architecture, the process is much the same. The architect breaks the business problem into chunks that represent discrete domains of activity. These chunks encapsulate the business rules and the data logic associated with a particular part of the business domain. For example, an architect might look at a business flow that needs to be carried out by code and realize that it needs both customer and product information.

TIP The presence of two discrete data domains is a good indication that multiple microservices are at play. How the two different parts of the business transaction interact usually becomes the service interface for the microservice.

Breaking down a business domain is an art form rather than black-and-white science. You can use the following guidelines for identifying and decomposing a business problem into microservice candidates:

- *Describe the business problem and notice the nouns you use to describe it.* Using the same nouns over and over in describing the problem is usually a good indication of a core business domain and an opportunity for a microservice. Examples of target nouns for the O-stock application might be something like contract, licenses, and assets.
- *Pay attention to the verbs.* Verbs highlight actions and often represent the natural contours of a problem domain. If you find yourself saying "transaction *X* needs to get data from thing *A* and thing *B*," that usually indicates that multiple services are at play.
  If you apply this approach to the O-stock application, you might look for statements like, "When Mike from desktop services sets up a new PC, he looks up the number of licenses available for software *X* and, if licenses are available, installs the software. He then updates the number of licenses used in his tracking spreadsheet." The key verbs here are looks and updates.
- *Look for data cohesion.* As you break your business problem down into discrete pieces, look for pieces of data that are highly related to one another. If suddenly, during the course of your conversation, you're reading or updating data that is radically different from what you've discussed, you potentially have another service candidate. Microservices must completely own their data.

Let's take these guidelines and apply them to a real-world problem, like the one for the O-stock software that is used for managing software assets. (We first mentioned this application in chapter 2.)

As a reminder, O-stock is Optima Growth's monolithic web application that is deployed to a Java EE application server residing within a customer's data center. Our goal is to tease apart the existing monolithic application into a set of services. To achieve this, we'll start by interviewing the users and some of the business stakeholders of the O-stock application, discussing how they interact with and use the application. Figure 3.1 summarizes and highlights a number of nouns and verbs of some of the conversations with different business customers.

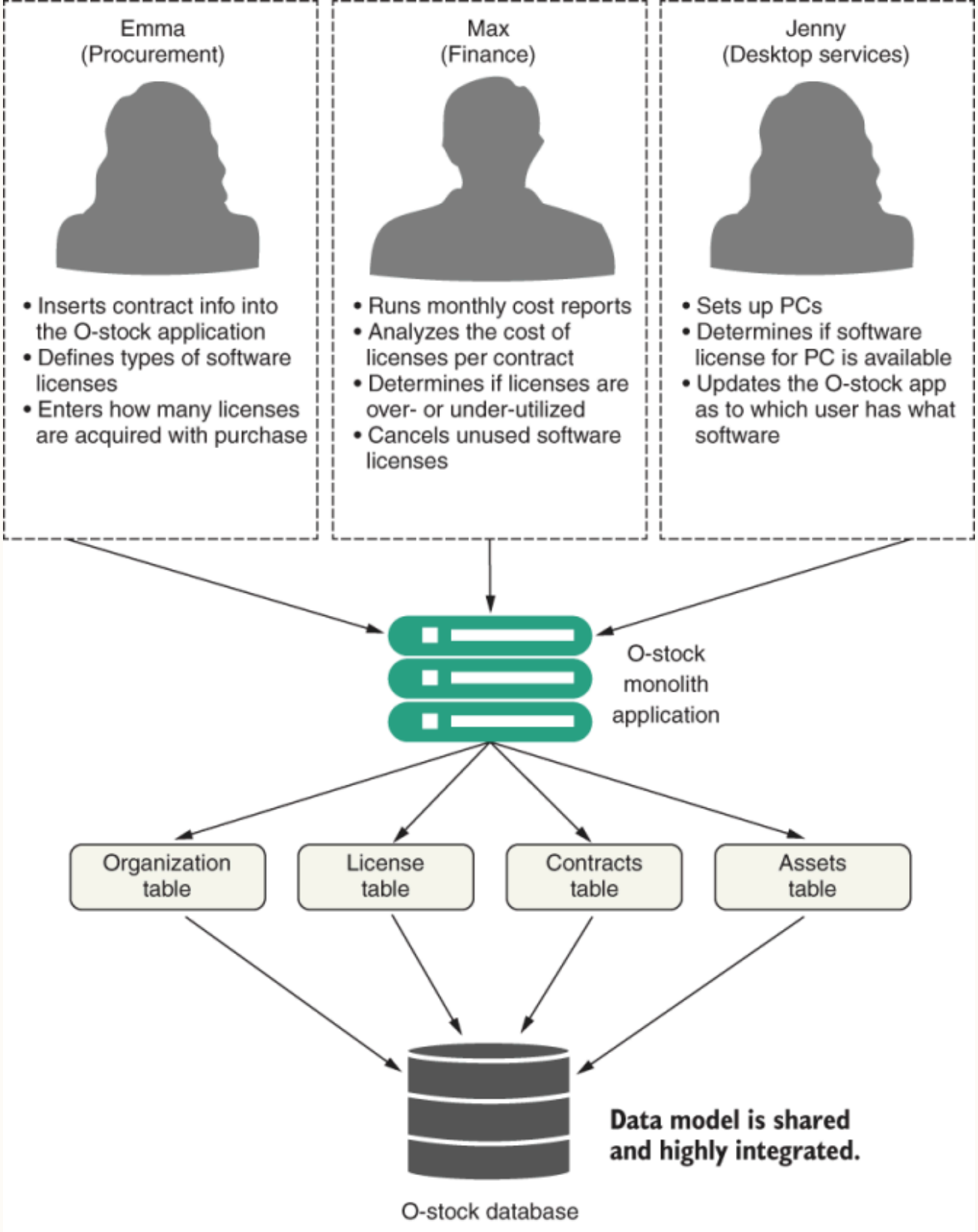


Figure 3.1 Summary of results from interviewing O-stock users to understand how they do their day-to-day work and interact with the application

Looking at how the users of O-stock interact with the application and answering the following questions, we can identify the data model for the application. By doing this, we can decompose the O-stock problem domain into microservice candidates.

- Where are we going to store the contract info managed by Emma?

- Where are we going to store and how are we going to manage the license information (cost, license type, license owner, and license contract)?
- Jenny sets up the licenses on the PCs. Where are we going to store the assets?
- Taking into consideration all the previously mentioned concepts, we can see that the license belongs to an organization that has several assets, right? So, where are we going to store the organization information?

Figure 3.2 shows a simplified data model based on the conversations with Optima Growth's customers. Based on the business interviews and the data model, the microservice candidates are *organization*, *license*, *contract*, and *assets*.

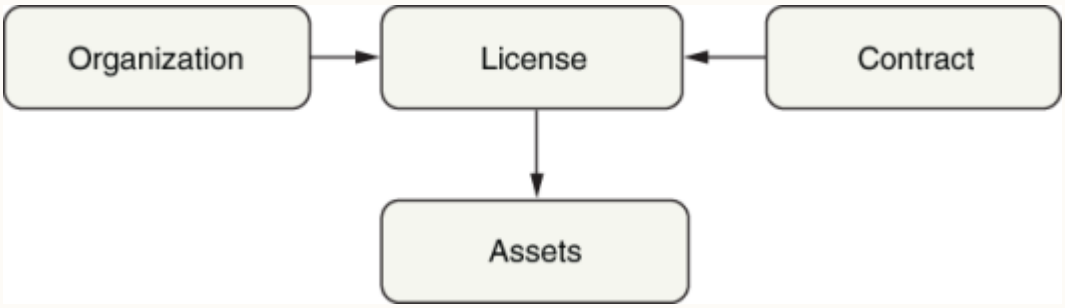


Figure 3.2 A simplified O-stock data model. An organization can have many licenses, the licenses can be applied to one or several assets, and each license has a contract.

1.2 Establishing service granularity

Once we have a simplified data model, we can begin the process of defining what microservices we are going to need in the application. As the data model in figure 3.2 illustrates, we can see the four potential microservices based on the following elements:

- Assets
- License
- Contract
- Organization

The goal is to take these major pieces of functionality and extract them into entirely self-contained units that we can build and deploy independently of each other. These units can optionally share or have individual databases. However, extracting services from the data model involves more than repackaging code into separate projects. It also involves teasing out the actual database tables the services will access and only allowing each service to access the tables in its specific domain. Figure 3.3 shows how the application code and the data model become "chunked" into individual pieces.

NOTE We created individual databases for each service, but you can also share databases between the services.

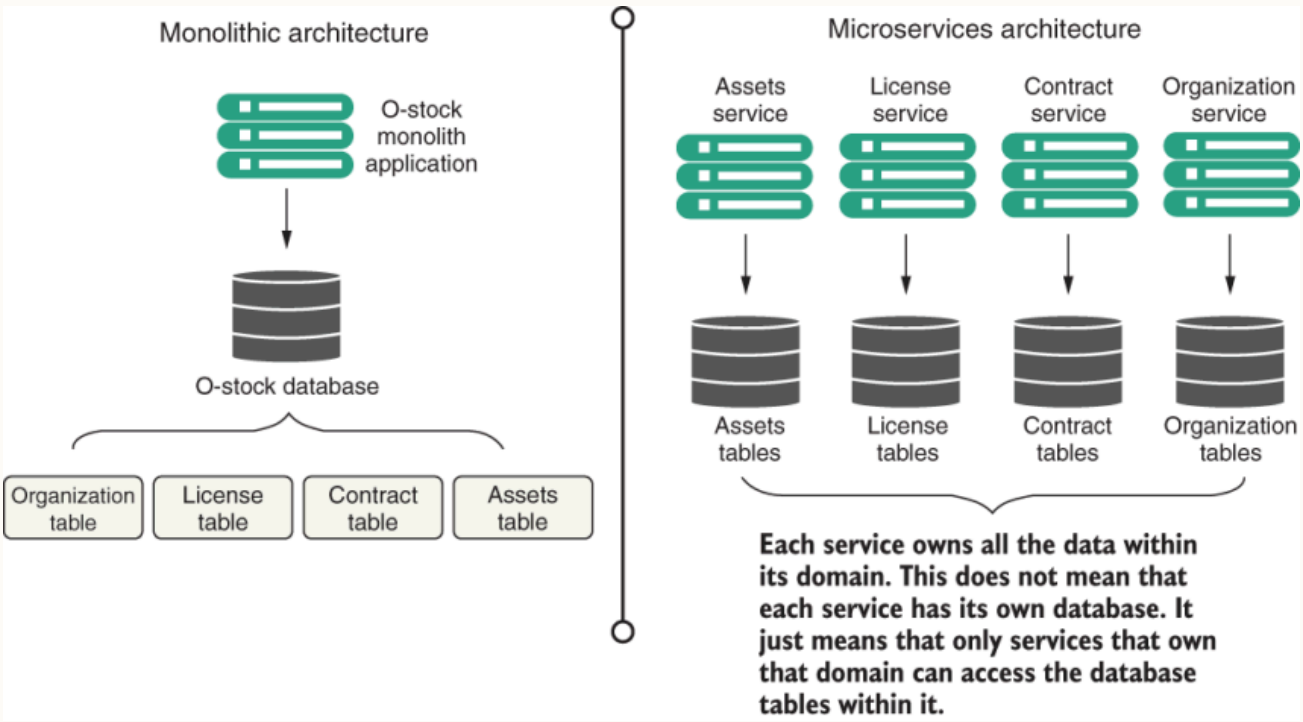


Figure 3.3 The O-stock application broken down from a monolithic application into smaller individual services that are deployed independently of one another

After we break down a problem domain into its discrete pieces, we will often find ourselves struggling to determine whether we've achieved the right level of *granularity* for our services. A microservice that is too coarse- or fine-grained will have several telltale attributes, which we'll discuss shortly.

When we build a microservice architecture, the question of granularity is essential. This is why we want to explain the following concepts to determine the correct answer to what is the right level of granularity:

- *It's better to start broad with our microservice and refactor to smaller services.* It is easy to go overboard when you begin your microservice journey and make everything a microservice. But decomposing the problem domain into small services often leads to premature complexity because microservices devolve into nothing more than fine-grained data services.
- *Focus first on how our services interact with one another.* This helps to establish the coarse-grained interfaces of your problem domain. It is easier to refactor from being too coarse-grained than from being too fine-grained.

- *Service responsibilities change over time as our understanding of the problem domain grows.* Often, a microservice gains responsibilities as new application functionalities are requested. What starts as a single microservice might grow into multiple services, with the original microservice acting as an orchestration layer for these new services and encapsulating their functionality from other parts of the application.

What smells as a bad microservice? How do you know whether your microservices are the right size? If a microservice is too coarse-grained, you'll likely see the following:

- *A service with too many responsibilities.* The general flow of the business logic in the service is complicated and seems to be enforcing an overly diverse array of rules.
- *A service that manages data across a large number of tables.* A microservice is the record for the data it manages. If you find yourself persisting data to multiple tables or reaching out to tables outside of the service database, this is a clue that the service is too big. We like to use the guideline that a microservice should own no more than three to five tables. Any more, and your service is likely to have too much responsibility.
- *A service with too many test cases.* Services can grow in size and responsibility over time. If you have a service that started with a small number of test cases and ended up with hundreds of unit and integration tests, you might need to refactor.

What about a microservice that's too fine-grained?

- *The microservices in one part of the problem domain breed like rabbits.* If everything becomes a microservice, composing business logic out of the services becomes complex and difficult. That's because the number of services needed to get a piece of work done grows tremendously. A common smell is when you have dozens of microservices in an application, and each service interacts with only a single database table.
- *Your microservices are heavily interdependent on one another.* You find that microservices in one part of the problem domain keep calling back and forth between each other to complete a single user request.
- *Your microservices become a collection of simple CRUD (Create, Replace, Update, Delete) services.* Microservices are an expression of business logic and not an abstraction layer over your data sources. If your microservices do nothing but CRUD-related logic, they're probably too fine-grained.

A microservices architecture should be developed with an evolutionary thought process, where you know that you aren't going to get the design right the first time. That is why it's better to start with your first set of services being more coarse-grained than fine-grained.

It is also essential not to be dogmatic with your design. You may run into physical constraints on your services. For example, you'll need to make an aggregation service that joins data together because two separate services will be too chatty, or no clear boundaries exist between the domain lines of a service. In the end, take a pragmatic approach and deliver rather than wasting time trying to get the design perfect and then have nothing to show for your efforts.

1.3 Defining the service interfaces

The last part of the architect's input is about defining how the microservices in your application will talk to one another. When building business logic with microservices, the interfaces for the services should be intuitive, and developers should get a rhythm of how all the services work in the application by fully understanding one or two of the services in the application. In general, we can use the following guidelines for implementing service interface design:

- *Embrace the REST philosophy.* This is one of the best practices (see appendix A), along with the Richardson Maturity Model (see the next sidebar). The REST approach to service is, at heart, the embracing of HTTP as the invocation protocol for the services, using standard HTTP verbs (GET, PUT, POST, and DELETE). Model your basic behaviors around these HTTP verbs.
- *Use URIs to communicate intent.* The URIs you use as endpoints for the service should describe the different resources in your problem domain and provide a basic mechanism for relationships of resources within it.
- *Use JSON for your requests and responses.* JSON is an extremely lightweight data-serialization protocol, and it's much easier to consume than XML.
- *Use HTTP status codes to communicate results.* The HTTP protocol has a rich body of standard response codes to indicate the success or failure of a service. Learn these status codes and, most importantly, use these consistently across all your services.

All the basic guidelines point to one thing: making your service interfaces easy to understand and consumable. You want a developer to sit down, look at the service interfaces, and start using them. If a microservice isn't easy to consume, developers will go out of their way to work around it and subvert the intention of the architecture.

3.2 When not to use microservices

We've spent this chapter talking about why microservices are a powerful architectural pattern for building applications. But we haven't touched on when you *shouldn't* use microservices to build your applications. Let's walk through these "should nots":

- Complexity when building distributed systems
- Virtual server or container sprawl
- Application type
- Data transactions and consistency

2.1 Complexity when building distributed systems

Because microservices are distributed and fine-grained (small), these introduce a level of complexity into your application that's not found in more monolithic applications. Microservice architectures require a high degree of operational maturity. Don't consider using microservices unless your organization is willing to invest in the automation and operational work (monitoring, scaling, and so on) that a highly distributed application needs to be successful.

2.2 Server or container sprawl

One of the most common deployment models for microservices is to have one microservice instance deployed in one container. In a large microservice-based application, you might end up with 50 to 100 servers or containers (usually virtual) that must be built and maintained in production alone. Even with the lower cost of running these services in the cloud, the operational complexity of managing and monitoring these services can be tremendous.

NOTE The flexibility of microservices has to be weighed against the cost of running all of those servers. You can also have different alternatives, such as considering functional developments like lambdas or adding more microservice instances on the same server.

2.3 Application type

Microservices are geared toward reusability and are extremely useful for building large applications that need to be highly resilient and scalable. This is one of the reasons why so many cloud-based companies have adopted microservices. If you're building small, departmental-level applications, or applications with a small user base, the complexity associated with building a distributed model like a microservice might generate more expense than it's worth.

2.4 Data transactions and consistency

As you begin looking at microservices, you need to think through the data usage patterns of your services and service consumers. A microservice wraps around and abstracts away a small number of tables and works well as a mechanism for performing "opera-

tional" tasks like creating, adding, and performing simple (non-complex) queries against a data store.

If your application needs to do complex data aggregation or transformation across multiple data sources, the distributed nature of microservices will make this work difficult. Your microservices will invariably take on too much responsibility and can also become vulnerable to performance problems.

3.3 The developer's tale: Building a microservice with Spring Boot and Java

In this section, we'll explore the developer's priorities in building the licensing microservice from the O-stock domain model.

NOTE We created the skeleton of the licensing service in the previous chapter. In case you didn't follow the code listings in that chapter, you can download the source code from **https://github.com/ihuaylupo/manning-smia/tree/master/chapter2.**

Over the next several sections, we'll

1. Implement a Spring Boot controller class for mapping an endpoint to expose the licensing service endpoints
2. Implement internationalization so that the messages can be adapted to different languages
3. Implement Spring HATEOAS to provide enough information so the user can interact with the server

3.1 Building the doorway into the microservice: The Spring Boot controller

Now that we've gotten the build script out of the way (see chapter 2) and have implemented a simple Spring Boot bootstrap class, you can write your first code that will do something. This code will be your controller class. In a Spring Boot application, the controller class exposes the service endpoints and maps the data from an incoming HTTP request to a Java method that processes the request.

Give it a REST

All the microservices in this book follow the Richardson Maturity Model (**http://mng.bz/JD5Z**). All the services you build will have the following characteristics:

- *Use HTTP/HTTPS as the invocation protocol for the service*—An HTTP endpoint exposes the service, and the HTTP protocol carries data to and from the service.
- *Map the behavior of the service to standard HTTP verbs*—REST emphasizes having services that map their behavior to the HTTP verbs POST, GET, PUT, and DELETE. These verbs map to the CRUD functions found in most services.
- *Use JSON as the serialization format for all data going to and from the service*—This isn't a hard-and-fast principle for REST-based microservices, but JSON has become "lingua franca" for serializing data that's submitted and returned by a microservice. You can use XML, but many REST-based applications make use of JavaScript and JSON. JSON is the native format for serializing and deserializing data consumed by JavaScript-based web front ends and services.
- *Use HTTP status codes to communicate the status of a service call*—The HTTP protocol uses a rich set of status codes to indicate the success or failure of a service. REST-based services take advantage of these HTTP status codes and other web-based infrastructures, such as reverse proxies and caches. These can be integrated with your microservices with relative ease.

HTTP is the language of the web. Using HTTP as the philosophical framework for building your service is key to building services in the cloud.

You'll find your first controller class located in src/main/java/com/optimagrowth/license/controller/LicenseController.java. The `LicenseController` class exposes four HTTP endpoints that will map to the POST, GET, PUT, DELETE verbs.

Let's walk through this controller class and see how Spring Boot provides a set of annotations that help to keep the effort needed to expose your service endpoints to a minimum and allow you to focus on building the business logic for the service. We'll start by looking at the basic controller class definition without any class methods in it just yet. The following listing shows the controller class for O-stock's licensing service.

Listing 3.1 Marking the `LicenseServiceController` as a Spring `RestController`

```
package com.optimagrowth.license.controller;

import java.util.Random;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
```

```
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;

import com.optimagrowth.license.model.License;

@RestController                                    ❶
@RequestMapping(value="v1/organization/            ❷
        {organizationId}/license")

public class LicenseController {
}
```

❶ Tells Spring Boot that this is a REST-based service and it will automatically serialize/deserialize service requests/responses via JSON.

❷ Exposes all the HTTP endpoints in this class with a prefix of /v1/organization/{organizationId}/license

We'll begin our exploration by looking at the `@RestController` annotation. The `@RestController` is a class-level Java annotation that tells the Spring container that this Java class will be used for a REST-based service. This annotation automatically handles the serialization of data passed into the services as either JSON or XML (by default, this class serializes returned data into JSON). Unlike the traditional Spring `@Controller` annotation, `@RestController` doesn't require you to return a `ResponseBody` class from your method in the controller class. This is all handled by the presence of the `@RestController` annotation, which includes the `@ResponseBody` annotation.

Why JSON for microservices?

We can use multiple protocols to send data back and forth between HTTP-based microservices. But JSON has emerged as the de facto standard for several reasons:

- *Compared to other protocols like XML-based SOAP (Simple Object Access Protocol), JSON is extremely lightweight.* You can express your data without having much textual overhead.
- *JSON is easily read and consumed by a human being.* This is an underrated quality for choosing a serialization protocol. When a problem arises, it's critical for developers to look at a chunk of JSON and to quickly process what's in it. The simplicity of the protocol makes this incredibly easy to do.

- *JSON is the default serialization protocol used in JavaScript.* Since the dramatic rise of JavaScript as a programming language and the equally dramatic rise of Single Page Internet Applications (SPIA) that rely heavily on JavaScript, JSON has become a natural fit for building REST-based applications, which is what the front-end web clients use to call services.
- Other mechanisms and protocols, however, are more efficient than JSON for communicating between services. The Apache Thrift (**http://thrift.apache.org**) framework allows you to build multilanguage services that can communicate with one another using a binary protocol. The Apache Avro protocol (**http://avro.apache.org**) is a data serialization protocol that converts data back and forth to a binary format between client and server calls. If you need to minimize the size of the data you're sending across the wire, we recommend you look at these protocols. But it has been our experience that using straight-up JSON in your microservices works effectively and doesn't interject another layer of communication to debug between your service consumers and service clients.

You can use the `@RequestMapping` annotation (the second annotation in listing 3.1) as a class-level and method-level annotation and to tell the Spring container the HTTP endpoint that the service is going to expose to the user. When you use `@RequestMapping` as the class-level annotation, you're establishing the root of the URL for all the other endpoints exposed by the controller. `@RequestMapping(value="v1/organization/ {organizationId}/ license")` uses the `value` attribute to establish the root of the URL for all endpoints exposed in the `Controller` class. All service endpoints exposed in this controller will start with

v1/organization/{organizationId}/license

as the root of their endpoint. The `{organizationId}` is a placeholder that indicates how you expect the URL to be parameterized with an `organizationId` passed in every call. The use of `organizationId` in the URL allows you to differentiate between the different customers who might use your service.

Before we add the first method to the controller, let's explore the model and the service class that we'll use in the services we're about to create. Listing 3.2 shows the POJO class that encapsulates the license data.

NOTE Encapsulation is one of the main principles of object-oriented programming, and in order to achieve encapsulation in Java, we must declare the variables of a class as pri-

vate and then provide public getters and setters to read and write the values of those variables.

Listing 3.2 Exploring the license model

```
package com.optimagrowth.license.model;

import lombok.Getter;
import lombok.Setter;
import lombok.ToString;

@Getter @Setter @ToString
public class License {                    (1)

    private int id;
    private String licenseId;
    private String description;
    private String organizationId;
    private String productName;
    private String licenseType;
}
```

(1) A Plain Old Java Object (POJO) that contains the license info

Lombok

Lombok is a small library that allows us to reduce the amount of boilerplate Java code written in the Java classes of our project. Lombok generates code such as getters and setters for string methods, constructors, and so forth.

In this book, we will use Lombok throughout the code examples in order to keep the code more readable, but we will not get into the details of how to use it. If you are interested in knowing more about Lombok, we highly recommend you look at the following Baeldung.com articles:

- https://www.baeldung.com/intro-to-project-lombok
- https://www.baeldung.com/lombok-ide

In case you want to install Lombok on the Spring Tools Suite 4, you must download and execute Lombok and link it to the IDE.

The following listing shows the service class that we'll use to develop the logic of the different services we are going to create on the controller class.

Listing 3.3 Exploring the `LicenseService` class

```
package com.optimagrowth.license.service;

import java.util.Random;

import org.springframework.stereotype.Service;
import org.springframework.util.StringUtils;

import com.optimagrowth.license.model.License;

@Service
public class LicenseService {

  public License getLicense(String licenseId, String organizationId){
    License license = new License();
    license.setId(new Random().nextInt(1000));
    license.setLicenseId(licenseId);
    license.setOrganizationId(organizationId);
    license.setDescription("Software product");
    license.setProductName("Ostock");
    license.setLicenseType("full");

    return license;
  }

  public String createLicense(License license, String organizationId){
    String responseMessage = null;
    if(license != null) {
      license.setOrganizationId(organizationId);
        responseMessage = String.format("This is the post and the
                        object is: %s", license.toString());
    }

    return responseMessage;
  }

  public String updateLicense(License license, String organizationId){
    String responseMessage = null;
    if (license != null) {
      license.setOrganizationId(organizationId);
      responseMessage = String.format("This is the put and
```

```
                the object is: %s", license.toString());
    }

    return responseMessage;
  }

  public String deleteLicense(String licenseId, String organizationId){
    String responseMessage = null;
    responseMessage = String.format("Deleting license with id %s for
                the organization %s",licenseId, organizationId);
    return responseMessage;

  }
}
```

This service class contains a set of dummy services returning hardcoded data to give you an idea of how the skeleton of a microservice should look. As you continue reading, you'll continue working on this service, and we'll delve further into how to structure it. For now, let's add the first method to the controller. This method implements the GET verb used in a REST call and returns a single `License` class instance as shown in the next code listing.

Listing 3.4 Exposing an individual GET HTTP endpoint

```
package com.optimagrowth.license.controller;

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;

import com.optimagrowth.license.model.License;
import com.optimagrowth.license.service.LicenseService;

@RestController
@RequestMapping(value="v1/organization/{organizationId}/license")
public class LicenseController {

@Autowired
  private LicenseService licenseService;
```

```
  @GetMapping(value="/{licenseId}")                    ❶
  public ResponseEntity<License> getLicense(
    @PathVariable("organizationId") String organizationId,
    @PathVariable("licenseId") String licenseId) {     ❷

    License license = licenseService
       .getLicense(licenseId,organizationId);          ❸
     return ResponseEntity.ok(license);
  }
}
```

❶ Get method to retrieve the license data

❷ Maps two parameters (organizationId and licenseId) from the URL to @GetMapping's parameters

❸ ResponseEntity represents the entire HTTP response.

The first thing we do in this listing was to annotate the `getLicense()` method with a `@GetMapping` annotation. Here we can use also the `@RequestMapping( value="/ {licenseId}", method = RequestMethod.GET )` annotation that passes two parameters (`value` and `method`) to the annotation. With a method-level `@GetMapping` annotation, we can build the following endpoint for the `getLicense()` method:

```
v1/organization/{organizationId}/license/{licenseId}
```

Why? If we go back and take a look at the top of the class, we specified a root-level annotation to match all HTTP requests coming to the controller: first, we added the root-level annotation value, and then, the method-level value. The second parameter of the annotation, `method` , specifies the HTTP verb used to match the method. In the `getLicense()` method, we match on the GET method as presented by the `RequestMethod.GET` enumeration.

The second thing to note in listing 3.4 is that we use the `@PathVariable` annotation in the parameter body of the `getLicense()` method. This annotation maps the parameter values passed in the incoming URL (as denoted by the `{parameterName}` syntax) to the parameters of your method. For the GET service in listing 3.4, we map two parameters from the URL (`organizationId` and `licenseId`) to two parameter-level variables in the method, like so:

```
@PathVariable("organizationId") String organizationId
@PathVariable("licenseId") String licenseId
```

Finally, let's examine the `ResponseEntity` return object. The `ResponseEntity` represents the entire HTTP response, including the status code, the headers, and the body. In the previous listing, it allows us to return the `License` object as the body and the `200(OK)` status code as the HTTP response of the service.

Now that you understand how to create an endpoint using the HTTP GET verb, let's continue by adding POST, PUT, and DELETE methods to create, update, and delete `License` class instances. The following listing shows how to do this.

Listing 3.5 Exposing individual HTTP endpoints

```
package com.optimagrowth.license.controller;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.DeleteMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.PutMapping;
import org.springframework.web.bind.annotation.RequestBody;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestMethod;
import org.springframework.web.bind.annotation.RestController;

import com.optimagrowth.license.model.License;
import com.optimagrowth.license.service.LicenseService;

@RestController
@RequestMapping(value="v1/organization/{organizationId}/license")
public class LicenseController {

  @Autowired
  private LicenseService licenseService;

  @RequestMapping(value="/{licenseId}",method = RequestMethod.GET)
  public ResponseEntity<License> getLicense(
      @PathVariable("organizationId") String organizationId,
      @PathVariable("licenseId") String licenseId) {

    License license = licenseService.getLicense(licenseId,
```

```
                                organizationId);
    return ResponseEntity.ok(license);
  }

  @PutMapping                                                    ❶
  public ResponseEntity<String> updateLicense(
      @PathVariable("organizationId")
      String organizationId,
      @RequestBody License request) {                            ❷
    return ResponseEntity.ok(licenseService.updateLicense(request,
                organizationId));
  }

  @PostMapping                                                   ❸
  public ResponseEntity<String> createLicense(
      @PathVariable("organizationId") String organizationId,
      @RequestBody License request) {
    return ResponseEntity.ok(licenseService.createLicense(request,
                organizationId));
  }

  @DeleteMapping(value="/{licenseId}")                           ❹
  public ResponseEntity<String> deleteLicense(
      @PathVariable("organizationId") String organizationId,
      @PathVariable("licenseId") String licenseId) {
    return ResponseEntity.ok(licenseService.deleteLicense(licenseId,
                organizationId));
  }
}
```

❶ Put method to update a license

❷ Maps the HTTP request body to a License object

❸ Post method to insert a license

❹ Delete method to delete a license

In listing 3.5, we first annotate the `updateLicense()` method with a method-level `@PutMapping` annotation. This annotation acts as a shortcut for the `@RequestMapping (method = RequestMethod.PUT)` annotation, which we haven't used yet.

The next thing to note is that we use the `@PathVariable` and the `@RequestBody` annotations in the parameter body of the `updateLicense()` method. `@RequestBody` maps

the `HTTPRequest` body to a transfer object (in this case, the `License` object). In the `updateLicense()` method, we map to the `createLicense` method from two parameters (one from the URL and the other from the `HTTPRequest` body) to the following two parameter-level variables:

```
@PathVariable("organizationId") String organizationId
@RequestBody License request
```

And finally, in listing 3.5 note that we use the `@PostMapping` and the `@DeleteMapping` annotations. The `@PostMapping` annotation is a method-level annotation that acts as a shortcut for

```
@RequestMapping(method = RequestMethod.POST)
```

`@DeleteMapping(value="/{licenseId}")` is also a method-level annotation and acts as a shortcut for

```
@RequestMapping(value="/{licenseId}",method = RequestMethod.DELETE)
```

Endpoint names matter

Before you get too far down the path of writing microservices, make sure that you (and potentially other teams in your organization) establish standards for the endpoints you want to exposed via your services. The URLs (Uniform Resource Locator) for the microservice should be used to clearly communicate the intent of the service, the resources the service manages, and the relationships that exist between the resources managed within the service. We've found the following guidelines useful for naming service endpoints:

- *Use clear URL names that establish what resource the service represents. Having a canonical format for defining URLs will help your API feel more intuitive and easier to use* —and be consistent in your naming conventions.
- *Use the URL to establish relationships between resources.* Often times, you'll have a parent-child relationship between resources within your microservices where the child doesn't exist outside the context of the parent. Hence, you might not have a separate microservice for the child. Use the URLs to express these relationships. If you find that your URLs tend to be long and nested, your microservice may be doing too much.

- *Establish a versioning scheme for URLs early.* The URL and its corresponding endpoints represent a contract between the service owner and the consumer of the service. One common pattern is to prepend all endpoints with a version number. Establish your versioning scheme early and stick to it. It's extremely difficult to retrofit versioning to URLS (for example, using /v1/ in the URL mappings) after you already have several consumers using them.

At this point, you have several services. From a command-line window, go to your project root directory, where you'll find pom.xml, and execute the following Maven command. (Figure 3.4 shows the expected output of this command.)

```
mvn spring-boot:run
```



Figure 3.4 Output showing that the licensing service successfully starts

Once the service is started, you can directly select the exposed endpoints. We highly recommend using a Chrome-based tool like Postman or cURL for calling the service. Figure 3.5 shows how to call the GET and DELETE services on an endpoint.

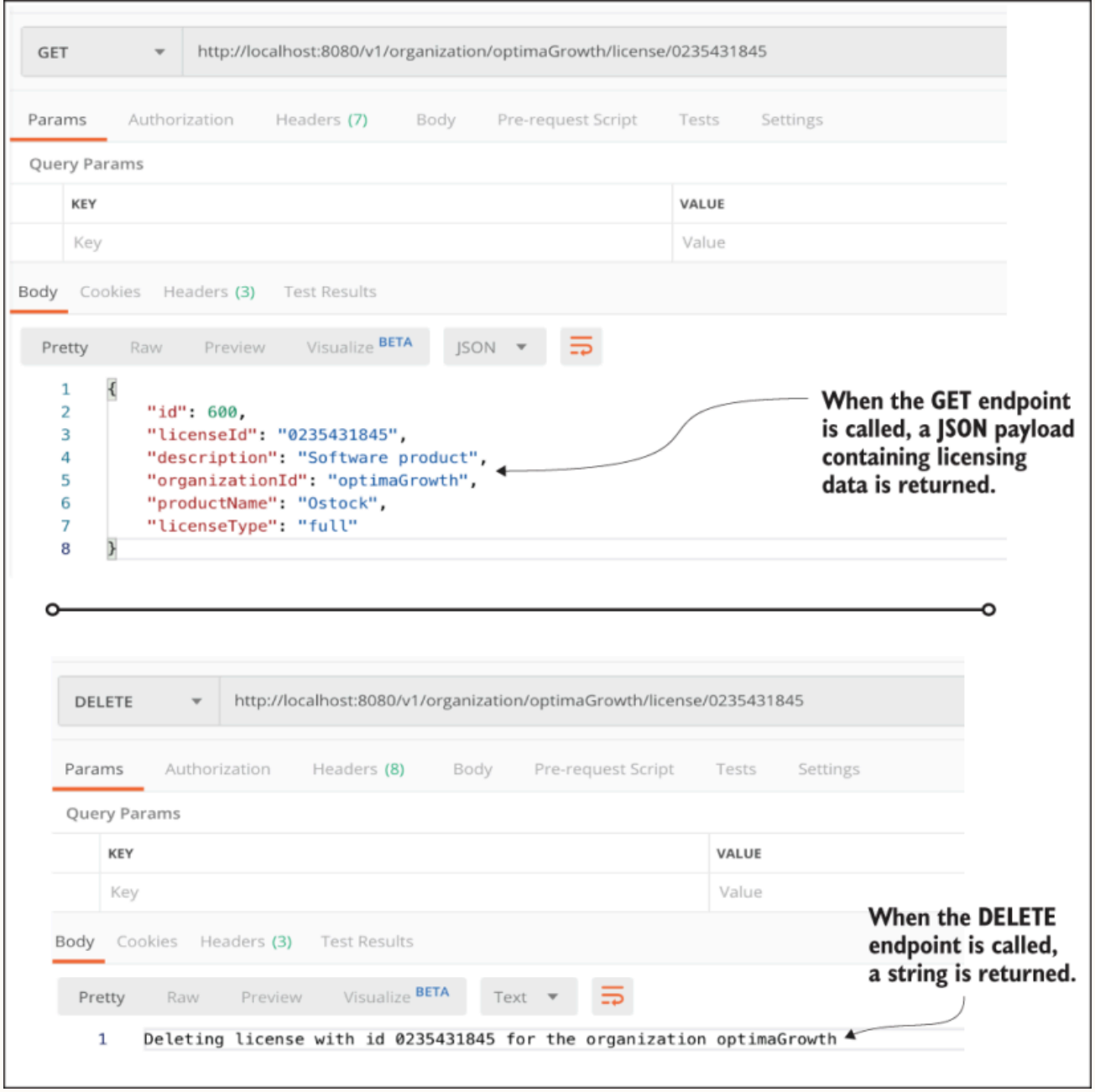


Figure 3.5 Your licensing GET and DELETE services called with Postman

Figure 3.6 shows how to call the POST and PUT services using the `http://localhost:8080/v1/organization/optimaGrowth/license` endpoint.

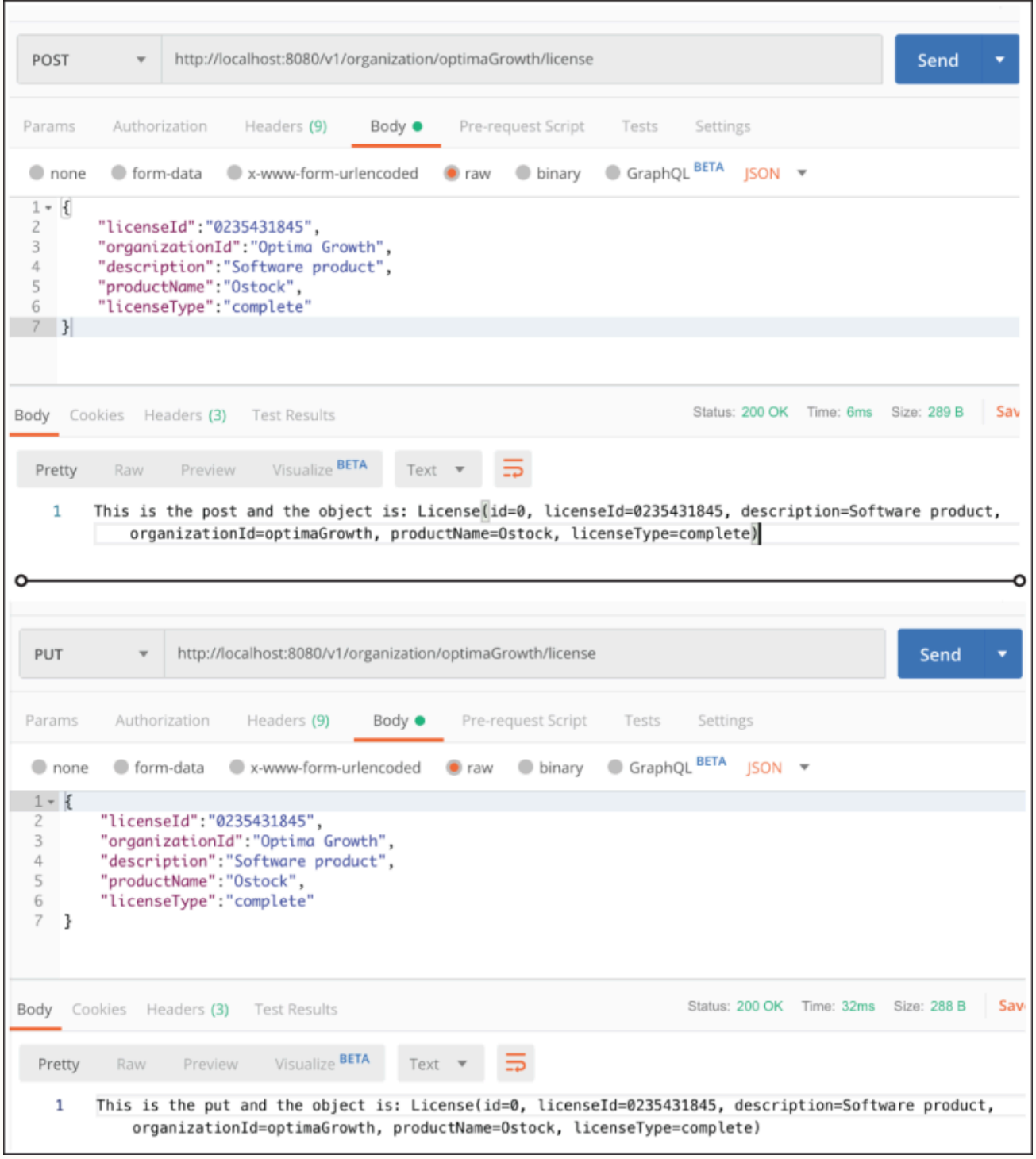


Figure 3.6 Your licensing GET and DELETE services called with Postman

Once we have implemented the methods for the PUT, DELETE, POST, and GET HTTP verbs, we can move on with internalization.

3.2 Adding internationalization into the licensing service

Internationalization is an essential requirement to enable your application to adapt to different languages. The main goal here is to develop applications that offer content in multiple formats and languages. In this section, we will explain how to add internalization to the licensing service we previously created.

First, we'll update the bootstrap class `LicenseServiceApplication.java` to create a `LocaleResolver` and a `ResourceBundleMessageSource` for our license service. The following listing shows how the bootstrap class should look.

Listing 3.6 Creating beans for the bootstrap class

```
package com.optimagrowth.license;

import java.util.Locale;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import org.springframework.context.annotation.Bean;
import org.springframework.context.support.ResourceBundleMessageSource;
import org.springframework.web.servlet.LocaleResolver;
import org.springframework.web.servlet.i18n.SessionLocaleResolver;

@SpringBootApplication
public class LicenseServiceApplication {

  public static void main(String[] args) {
    SpringApplication.run(LicenseServiceApplication.class, args);
  }

  @Bean
  public LocaleResolver localeResolver() {
    SessionLocaleResolver localeResolver = new SessionLocaleResolver();
    localeResolver.setDefaultLocale(Locale.US);                    (1)
    return localeResolver;
  }
  @Bean
  public ResourceBundleMessageSource messageSource() {
    ResourceBundleMessageSource messageSource =
                        new ResourceBundleMessageSource();
    messageSource.setUseCodeAsDefaultMessage(true);                (2)

    messageSource.setBasenames("messages");                        (3)
    return messageSource;
  }
}
```

(1) Sets US as the default locale

(2) Doesn't throw an error if a message isn't found, instead it returns the message code

❸ Sets the base name of the languages properties files

The first thing to note in listing 3.6 is that we set `Locale.US` as the default locale. If we don't set a `Locale` when we retrieve the message, the `messageSource` will use the default locale set as the `LocaleResolver`. Next, note the following call:

```
messageSource.setUseCodeAsDefaultMessage(true)
```

When a message is not found, this option returns the message code `'license.creates.message'` instead of an error like this one:

```
"No message found under code 'license.creates.message' for locale 'es'
```

Finally, the `messageSource.setBasenames("messages")` call sets `messages` as the base name of the message source file. For example, if we were in Italy, we would use the `Locale.IT`, and we would have a file called messages_it.properties. In case we don't find a message in a specific language, the message source will search on the default message file called messages.properties.

Now, let's configure the messages. For this example, we will use English and Spanish messages. To achieve this, we need to create the following files under the /src/main/resources source folder:

- messages_en.properties
- messages_es.properties
- messages.properties

The following two listings show how the messages_en.properties and the messages_es.properties files should look.

Listing 3.7 Exploring the messages_en.properties file

```
license.create.message = License created %s
license.update.message = License %s updated
license.delete.message = Deleting license with
            id %s for the organization %s
```

Listing 3.8 Exploring the messages_es.properties file

```
license.create.message = Licencia creada %s
license.update.message = Licencia %s creada
license.delete.message = Eliminando licencia con
            id %s para la organization %s license
```

Now that we've implemented the messages and the `@Beans` annotation, we can update the code in our controller or service to call the message resource. The following listing shows how this is done.

Listing 3.9 Updating the service to look for the messages on the `MessageSource`

```
@Autowired
MessageSource messages;

public String createLicense(License license,
                String organizationId,
                Locale locale){                          (1)
    String responseMessage = null;
    if (license != null) {
      license.setOrganizationId(organizationId);
      responseMessage = String.format(messages.getMessage(
                "license.create.message", null,locale),
                license.toString());                     (2)
    }

    return responseMessage;
}

public String updateLicense(License license, String organizationId){
    String responseMessage = null;
    if (license != null) {
      license.setOrganizationId(organizationId);
      responseMessage = String.format(messages.getMessage(
                "license.update.message", null, null),
                license.toString());                     (3)
    }

    return responseMessage;
}
```

(1) Receives the Locale as a method parameter

(2) Sets the received locale to retrieve the specific message

❸ Sends a null locale to retrieve the specific message

There are three important things to highlight from the code in listing 3.9. The first is that we can receive the `Locale` from the `Controller` itself. The second is that we can call the `messages.getMessage("license.create.message",null,locale)` using the locale we received by parameters, and the third thing to note is that we can call the `messages.getMessage("license.update.message", null, null)` without sending any locale. In this particular scenario, the application will use the default locale we previously defined in the bootstrap class. Now let's update our `createLicense()` method on the controller to receive the language from the request `Accept-Language` header with this code:

```
@PostMapping
public ResponseEntity<String> createLicense(
    @PathVariable("organizationId") String organizationId,
    @RequestBody License request,
    @RequestHeader(value = "Accept-Language",required = false)
                Locale locale){
 return ResponseEntity.ok(licenseService.createLicense(
                    request, organizationId, locale));
}
```

A few things to note from this code is that here we use the `@RequestHeader` annotation. The `@RequestHeader` annotation maps method parameters with request header values. In the `createLicense()` method, we retrieve the locale from the request header `Accept-Language`. This service parameter is not required, so if it's not specified, we will use the default locale. Figure 3.7 shows you how to send the `Accept-Language` request header from Postman.

NOTE There isn't a well-defined rule on how to use `locale`. Our recommendation is to analyze your architecture and select the option that is most suitable for you. For example, if the front-end application handles the locale, then receiving the locale as a parameter in the controller method is the best option. But if you are managing the locale in the backend, you can use a default locale.

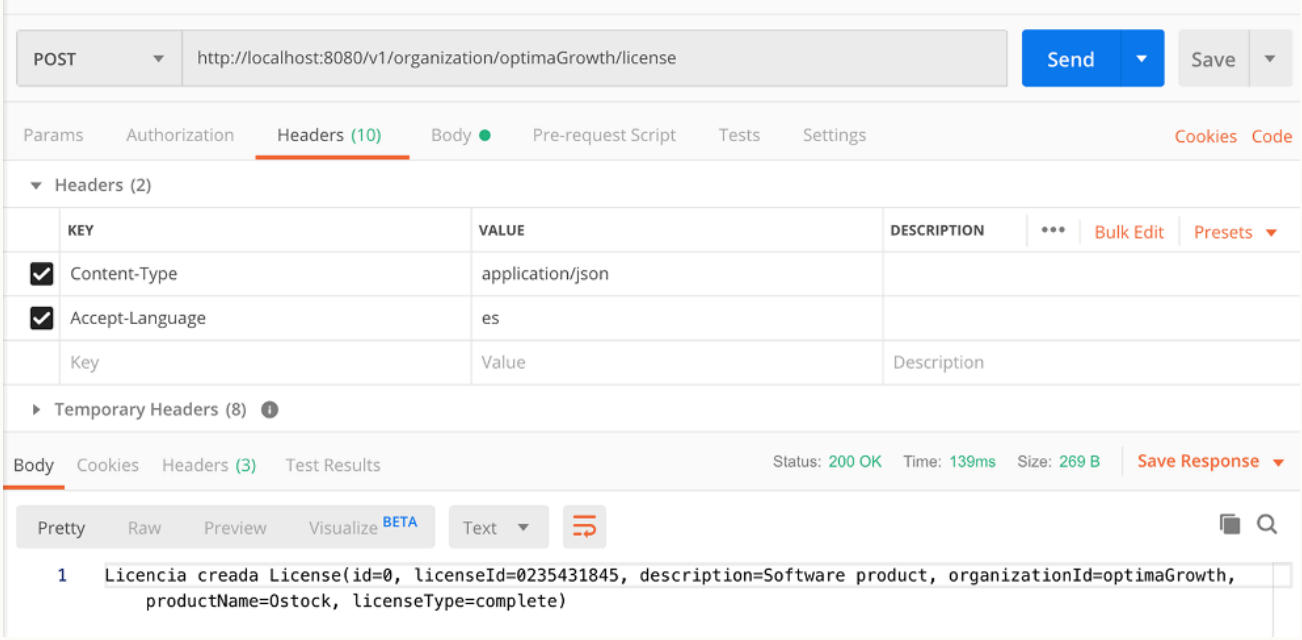


Figure 3.7 Setting the `Accept-Language` header in the POST create license service

3.3 Implementing Spring HATEOAS to display related links

HATEOAS stands for Hypermedia as the Engine of Application State. Spring HATEOAS is a small project that allows us to create APIs that follow the HATEOAS principle of displaying the related links for a given resource. The HATEOAS principle states that an API should provide a guide to the client by returning information about possible next steps with each service response. This project isn't a core or a must-have feature, but if you want to have a complete guide for all of the API services of a given resource, it is an excellent option.

With Spring HATEOAS, you can quickly create model classes for links to resource representation models. It also provides a link builder API to create specific links that point to Spring MVC controller methods. The following code snippet shows an example of how HATEOAS should look for the license service:

```
"_links": {
   "self" : {
      "href" : "http://localhost:8080/v1/organization/
               optimaGrowth/license/0235431845"
   },
   "createLicense" : {
      "href" : "http://localhost:8080/v1/organization/
              optimaGrowth/license"
   },
   "updateLicense" : {
      "href" : "http://localhost:8080/v1/organization/
               optimaGrowth/license"
```

```
    },
    "deleteLicense" : {
       "href" : "http://localhost:8080/v1/organization/
              optimaGrowth/license/0235431845"
    }
}
```

In this section, we will show you how to implement Spring HATEOAS in the license service. The first thing we must do to send the links related to a resource in the response is to add the HATEOAS dependency into our pom.xml file, like this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-hateoas</artifactId>
</dependency>
```

Once we have the dependency, we need to update the `License` class in order to extend from `RepresentationModel<License>`. The following listing shows how to do this.

Listing 3.10 Extending from `RepresentationModel`

```
package com.optimagrowth.license.model;

import org.springframework.hateoas.RepresentationModel;

import lombok.Getter;
import lombok.Setter;
import lombok.ToString;

@Getter @Setter @ToString
public class License extends RepresentationModel<License> {

  private int id;
  private String licenseId;
  private String description;
  private String organizationId;
  private String productName;
  private String licenseType;

}
```

`RepresentationModel<License>` gives us the ability to add links to the `License` model class. Now that we have everything set up, let's create the HATEOS configuration to retrieve the links for the `LicenseController` class. The next listing shows how this is done. For this example, we're only going to change the `getLicense()` method in the `LicenseController` class.

Listing 3.11 Adding links to the `LicenseController`

```
@RequestMapping(value="/{licenseId}",method = RequestMethod.GET)
public ResponseEntity<License> getLicense(
        @PathVariable("organizationId") String organizationId,
        @PathVariable("licenseId") String licenseId) {

    License license = licenseService.getLicense(licenseId,
                                organizationId);
    license.add(linkTo(methodOn(LicenseController.class)
        .getLicense(organizationId, license.getLicenseId()))
        .withSelfRel(),
        linkTo(methodOn(LicenseController.class)
        .createLicense(organizationId, license, null))
        .withRel("createLicense"),
        linkTo(methodOn(LicenseController.class)
        .updateLicense(organizationId, license))
        .withRel("updateLicense"),
        linkTo(methodOn(LicenseController.class)
        .deleteLicense(organizationId, license.getLicenseId()))
        .withRel("deleteLicense"));
    return ResponseEntity.ok(license);
}
```

The method `add()` is a method of the `RepresentationModel`. The `linkTo` method inspects the `License` controller class and obtains the root mapping, and the `methodOn` method obtains the method mapping by doing a dummy invocation of the target method. Both methods are static methods of `org.springframework.hateoas.server.mvc.WebMvcLinkBuilder`. `WebMvcLinkBuilder` is a utility class for creating links on the controller classes. Figure 3.8 shows the links on the response body of the `getLicense()` service. To retrieve those, you must call the GET HTTP method.

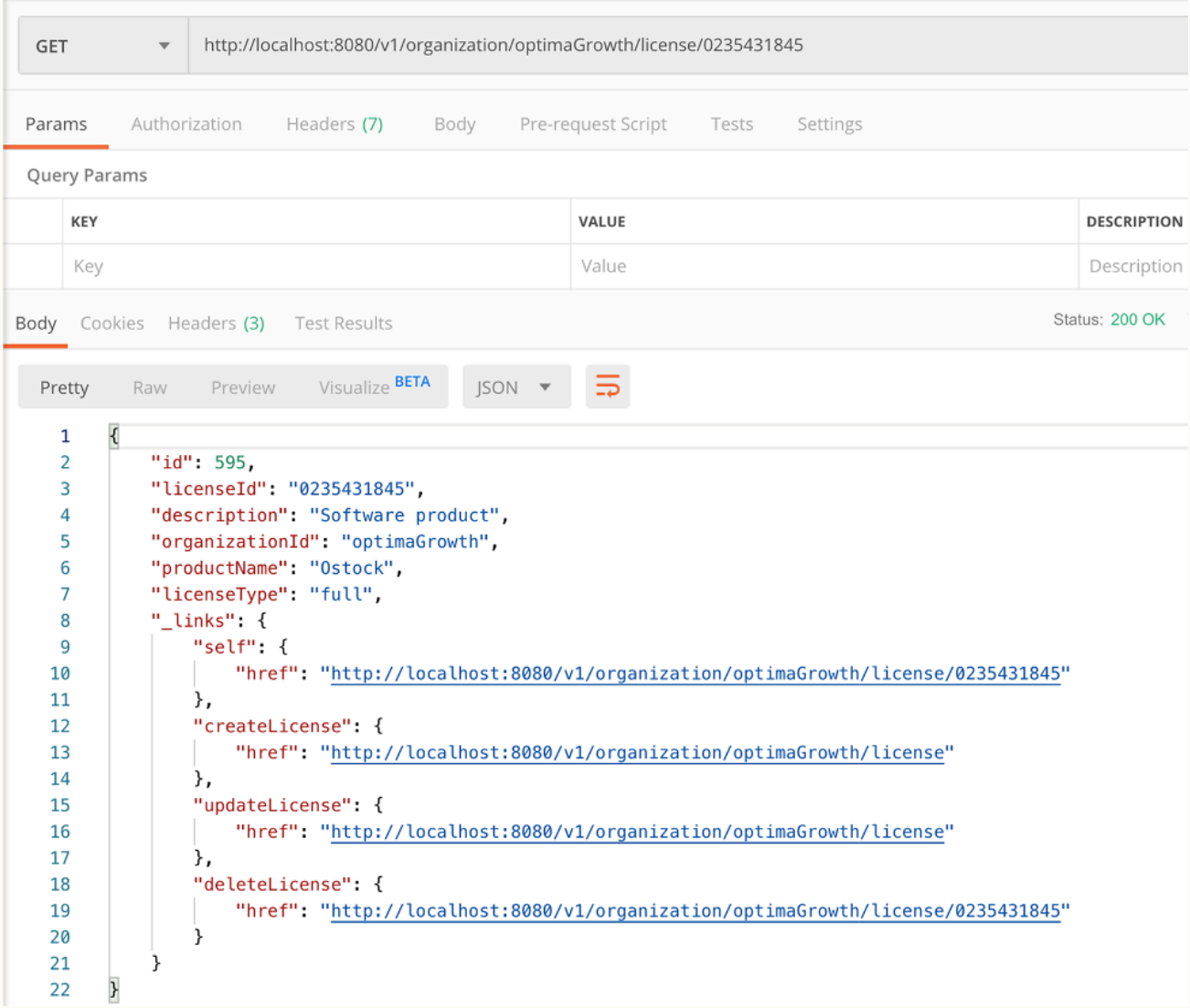


Figure 3.8 HATEOAS links on the response body of the HTTP GET license service

At this point, you have a running skeleton of a service. But from a development perspective, this service isn't complete. A good microservice design doesn't eschew segregating the service into well-defined business logic and data access layers. As you progress in later chapters, you'll continue to iterate on this service and delve further into how to structure it. For now, let's switch to the final perspective: exploring how a DevOps engineer would operationalize the service and package it for deployment to the cloud.

3.4 The DevOps story: Building for the rigors of runtime

While DevOps is a rich and emerging IT field, for the DevOps engineer, the design of the microservice is all about managing the service after it goes into production. Writing the code is often the easy part. Keeping it running is the hard part. We'll start our microservice development effort with four principles and build on these later in the book:

- *A microservice should be self-contained.* It should also be independently deployable with multiple instances of the service being started up and torn down with a single software artifact.

- *A microservice should be configurable.* When a service instance starts up, it should read the data it needs to configure itself from a central location or have its configuration information passed on as environment variables. No human intervention should be required to configure the service.
- *A microservice instance needs to be transparent to the client.* The client should never know the exact location of a service. Instead, a microservice client should talk to a service discovery agent. That allows the application to locate an instance of a microservice without having to know its physical location.
- *A microservice should communicate its health.* This is a critical part of your cloud architecture. Microservice instances will fail, and discovery agents need to route around bad service instances. In this book, we'll use Spring Boot Actuator to display the health of each microservice.

These four principles expose the paradox that can exist with microservice development. Microservices are smaller in size and scope, but their use introduces more moving parts in an application because microservices are distributed and run independently of each other in their own containers. This introduces a high degree of coordination and more opportunities for failure points in the application.

From a DevOps perspective, you must address the operational needs of a microservice up front and translate these four principles into a standard set of lifecycle events that occur every time a microservice is built and deployed to an environment. The four principles can be mapped to the following operational lifecycles. Figure 3.9 shows how these four steps fit together.

- *Service assembly*—How you package and deploy your service to guarantee repeatability and consistency so that the same service code and run time are deployed exactly the same way.
- *Service bootstrapping*—How you separate your application and environment-specific configuration code from the run-time code so that you can start and deploy a microservice instance quickly in any environment without human intervention.
- *Service registration/discovery*—When a new microservice instance is deployed, how you make the new service instance discoverable by other application clients.
- *Service monitoring*—In a microservice environment, it's common for multiple instances of the same service to be running due to high availability needs. From a DevOps perspective, you need to monitor microservice instances and ensure that any faults are routed around failing service instances, and that these are taken down.

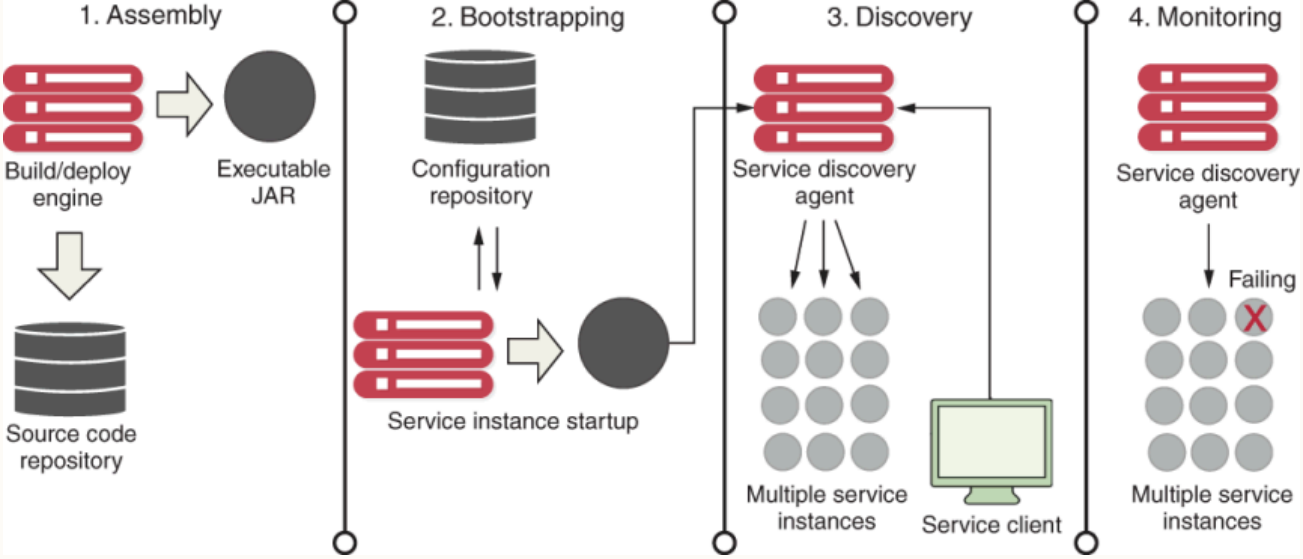


Figure 3.9 A microservice goes through multiple steps in its lifecycle.

4.1 Service assembly: Packaging and deploying your microservices

From a DevOps perspective, one of the key concepts behind a microservice architecture is that multiple instances of a microservice can be deployed quickly in response to a changed application environment (for example, a sudden influx of user requests, problems within the infrastructure, and so on). To accomplish this, a microservice needs to be packaged and installable as a single artifact with all of its dependencies defined within it. These dependencies must also include the run-time engine (for example, an HTTP server or application container) that hosts the microservice.

The process of consistently building, packaging, and deploying is the service assembly (step 1 in figure 3.9). Figure 3.10 shows additional details about this step.

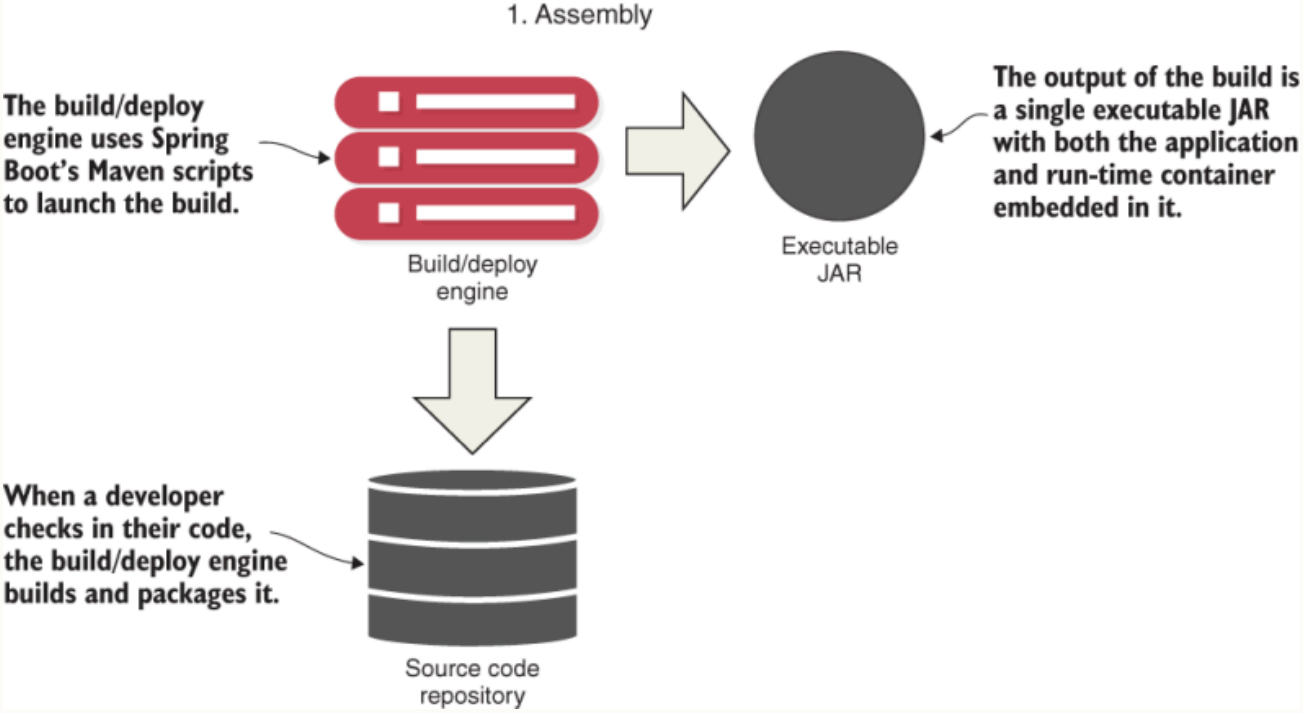


Figure 3.10 In the service assembly step, source code is compiled and packaged with its run-time engine.

Fortunately, almost all Java microservice frameworks will include a run-time engine that you can package and deploy with the code. For instance, in the Spring Boot example in figure 3.10, Maven and Spring Boot build an executable Java JAR file that has an embedded Tomcat engine built right into the JAR. In the following command-line example, we build the licensing service as an executable JAR and then start the JAR from the command line:

```
mvn clean package && java -jar target/licensing-service-0.0.1-SNAPSHOT.jar
```

For certain operations teams, the concept of embedding a run-time environment right in the JAR file is a major shift in the way they think about deploying applications. In a traditional Java web-based application, the application is deployed to an application server. This model implies that the application server is an entity in and of itself and would often be managed by a team of system administrators who oversee the server's configuration independently of the applications being deployed to them.

This separation of the application server configuration from the application introduces failure points in the deployment process. This is because in many organizations, the configuration of the application servers isn't kept under source control and is managed through a combination of the user interface and home-grown management scripts. It's too easy for configuration drift to creep into the application server environment and suddenly cause what on the surface appears to be random outages.

4.2 Service bootstrapping: Managing configuration of your microservices

Service bootstrapping (step 2 in figure 3.9) occurs when the microservice first starts and needs to load its application configuration information. Figure 3.11 provides more context for the bootstrapping process.

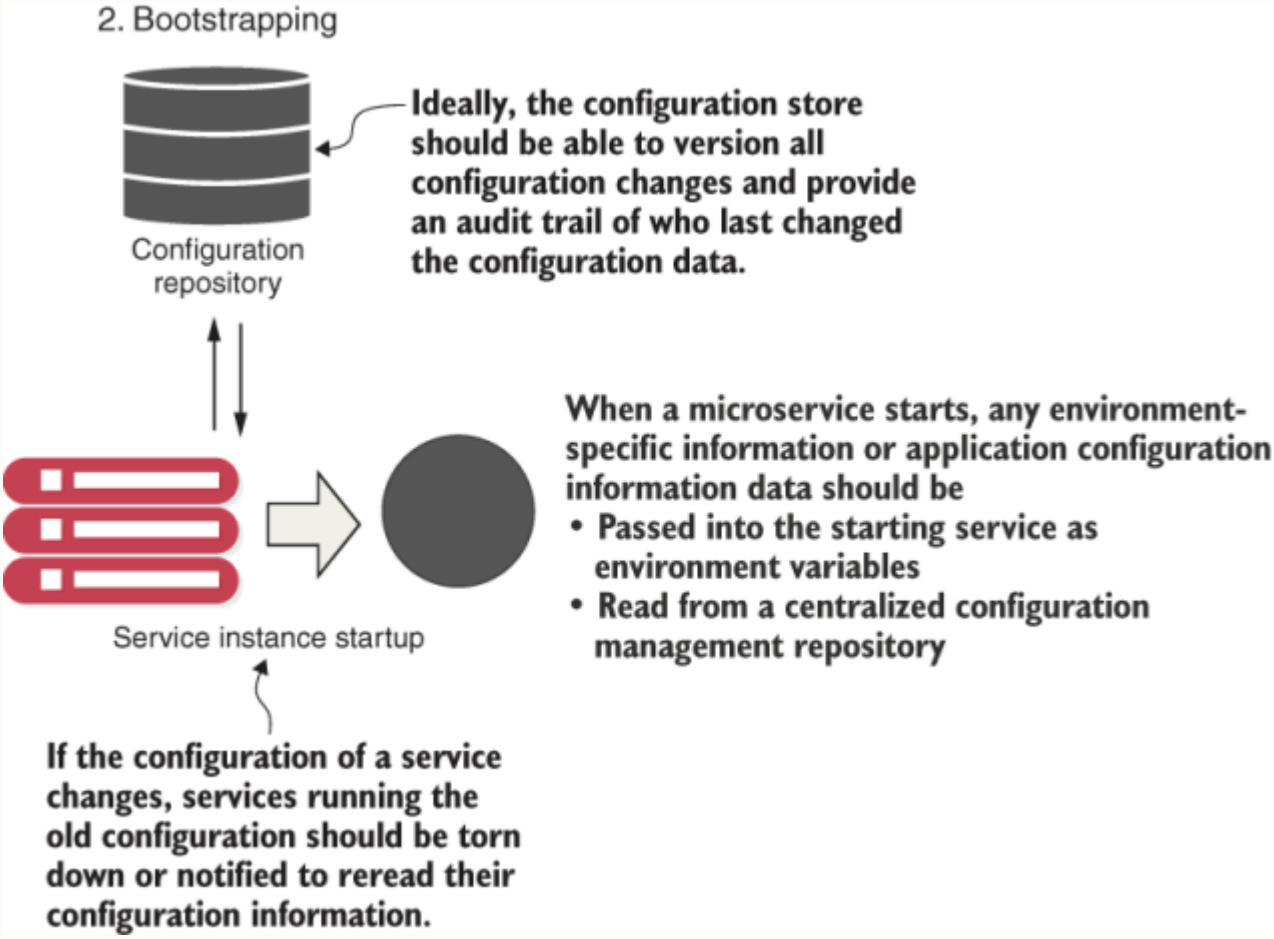


Figure 3.11 As a service starts (bootstraps), it reads its configuration from a central repository.

As any application developer knows, there will be times when you need to make the runtime behavior of the application configurable. Usually this involves reading your application configuration data from a property file deployed with the application or reading the data out of a data store like a relational database.

Microservices often run into the same type of configuration requirements. The difference is that in a microservice application running in the cloud, you might have hundreds or even thousands of microservice instances running. Further complicating this is that the services might be spread across the globe. With a high number of geographically dispersed services, it becomes unfeasible to redeploy your services to pick up new configuration data. Storing the data in a data store external to the service solves this problem. But microservices in the cloud offer a set of unique challenges:

- Configuration data tends to be simple in structure and is usually read frequently and written infrequently. Relational databases are overkill in this situation because they're designed to manage much more complicated data models than a simple set of key-value pairs.
- Because the data is accessed on a regular basis but changes infrequently, the data must be readable with a low level of latency.
- The data store has to be highly available and close to the services reading the data. A configuration data store can't go down completely because it would become a single point of failure for your application.

In chapter 5, we'll show how to manage your microservice application configuration data using things like a simple key-value data store.

4.3 Service registration and discovery: How clients communicate with your microservices

From a microservice consumer perspective, a microservice should be location-transparent because in a cloud-based environment, servers are ephemeral. *Ephemeral* means that the servers that a service is hosted on usually have shorter lives than a service running in a corporate data center. Cloud-based services can be started and torn down quickly with an entirely new IP address assigned to the server on which the services are running.

By insisting that services are treated as short-lived disposable objects, microservice architectures can achieve a high degree of scalability and availability by having multiple instances of a service running. Service demand and resiliency can be managed as quickly as the situation warrants. Each service has a unique and impermanent IP address assigned to it. The downside to ephemeral services is that with services constantly coming up and down, managing a large pool of these services manually or by hand is an invitation to an outage.

A microservice instance needs to register itself with a third-party agent. This registration process is called *service discovery* (see step 3 in figure 3.9, then see figure 3.12 for details on this process). When a microservice instance registers with a service discovery agent, it tells the discovery agent two things: the physical IP address (or domain address of the service instance) and a logical name that an application can use to look up the service. Certain service discovery agents also require a URL sent back to the registering service, which can be used by the service discovery agent to perform health checks. The service client then communicates with the discovery agent to look up the service's location.

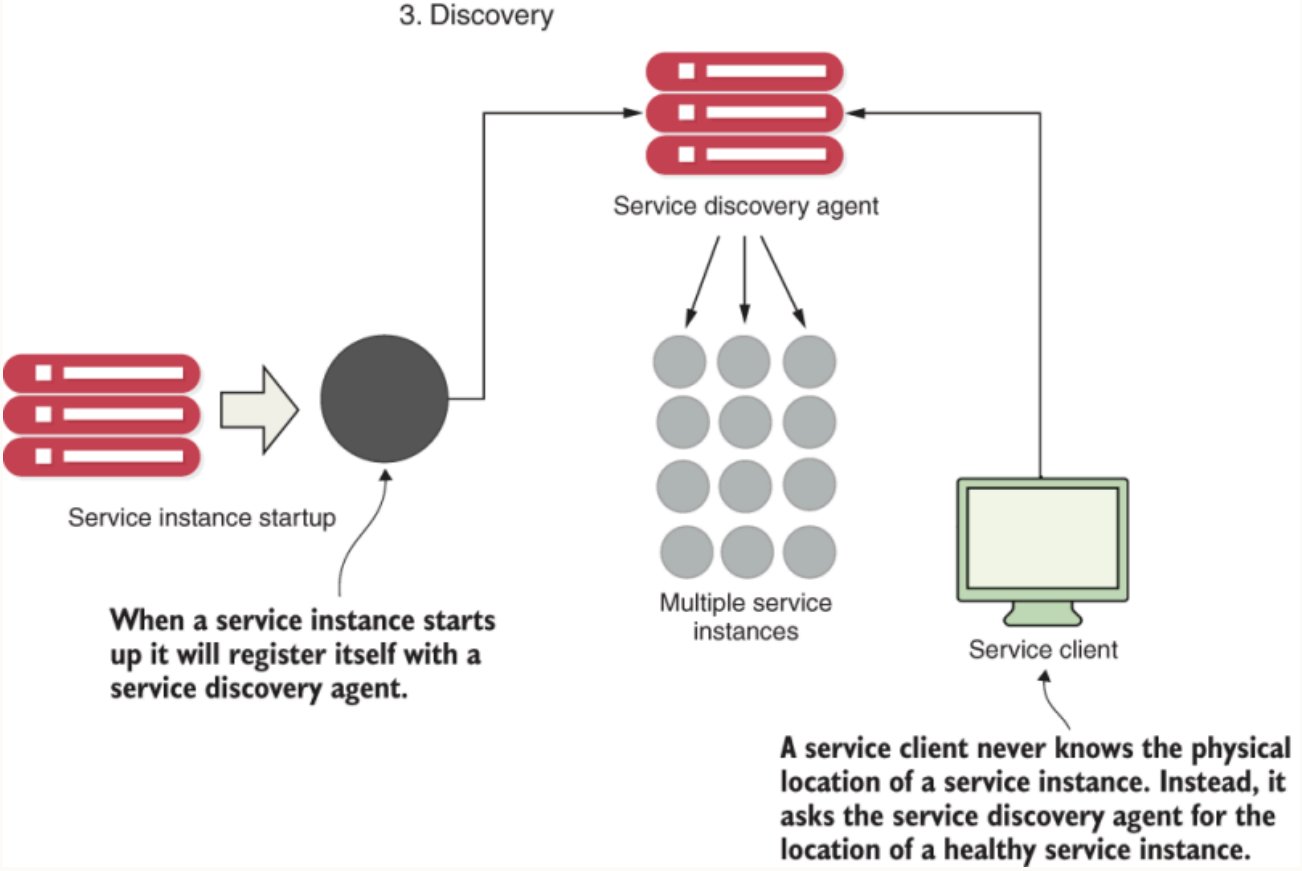


Figure 3.12 A service discovery agent abstracts away the physical location of a service.

4.4 Communicating a microservice's health

A service discovery agent doesn't act only as a traffic cop that guides the client to the location of the service. In a cloud-based microservice application, you'll often have multiple instances of a running service. Sooner or later, one of those service instances will fail. The service discovery agent monitors the health of each service instance registered with it and removes any failing service instances from its routing tables to ensure that clients aren't sent a service instance that has failed.

After a microservice comes up, the service discovery agent will continue to monitor and ping the health check interface to ensure that that service is available. This is step 4 in figure 3.9; figure 3.13 provides context for this step.

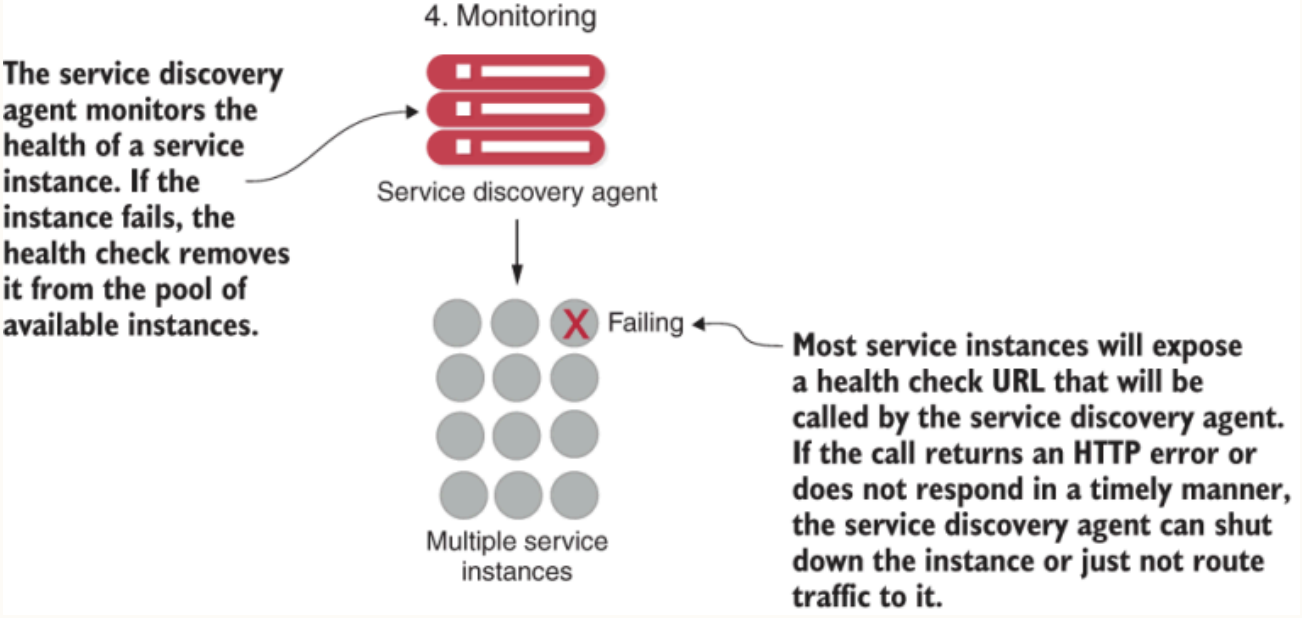


Figure 3.13 The service discovery agent uses the exposed health URL to check the microservice state.

By building a consistent health check interface, you can use cloud-based monitoring tools to detect problems and respond to these appropriately. If the service discovery agent discovers a problem with a service instance, it can take corrective action such as shutting down the ailing instance or bringing additional service instances up.

In a microservice environment that uses REST, the simplest way to build a health check interface is to expose an HTTP endpoint that can return a JSON payload and HTTP status code. In a non-Spring Boot-based microservice, it's often the developer's responsibility to write an endpoint that returns the health of the service.

In Spring Boot, exposing an endpoint is trivial and involves nothing more than modifying your Maven build file to include the Spring Actuator module. Spring Actuator provides out-of-the-box operational endpoints that help you understand and manage the health of your service. To use Spring Actuator, you need to include the following dependencies in your Maven build file:

```
<dependency>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

If you reach the `http://localhost:8080/actuator/health` endpoint on the licensing service, you should see health data returned. Figure 3.14 provides an example of the data returned.

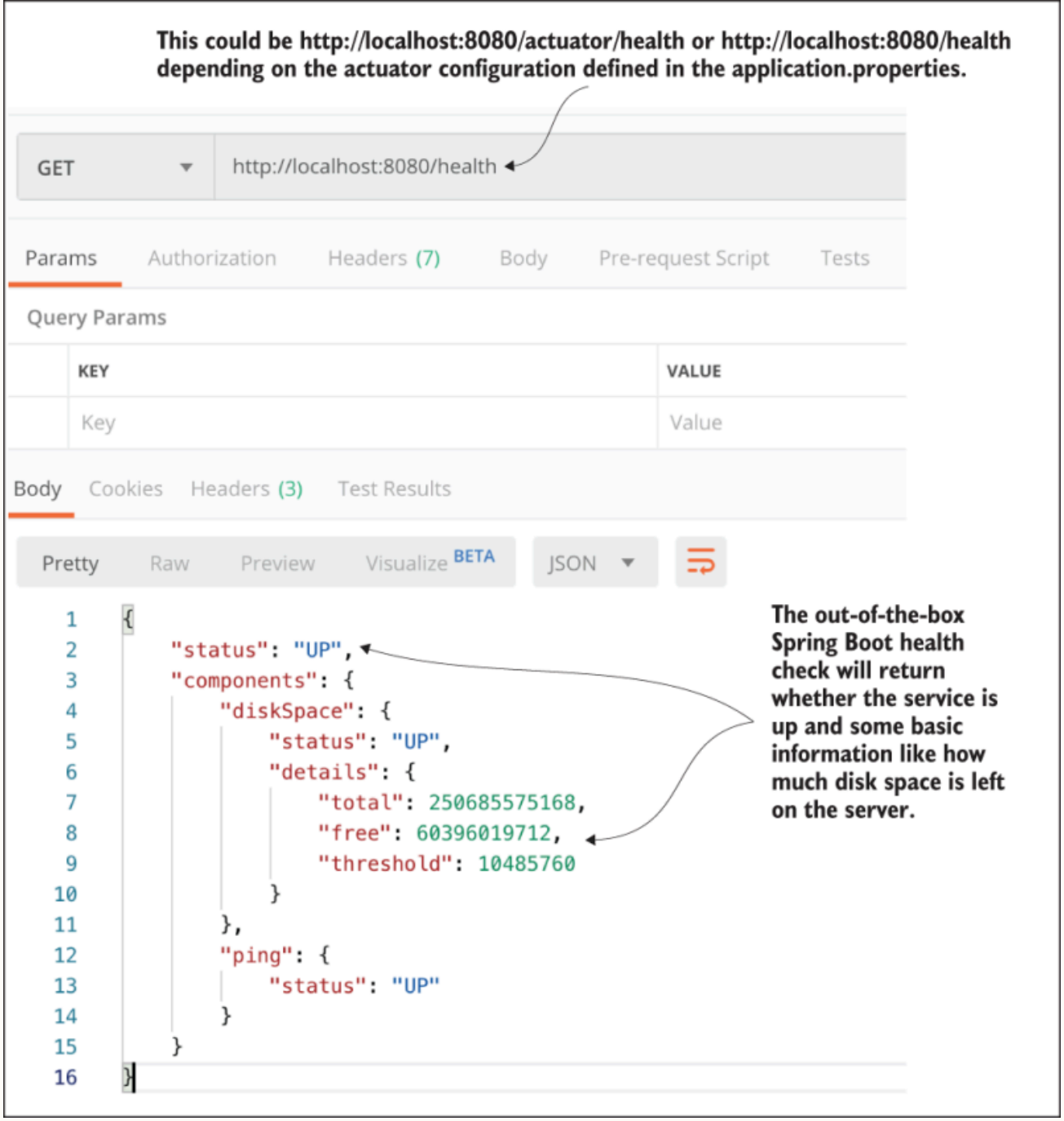


Figure 3.14 A health check on each service instance allows monitoring tools to determine if the service instance is running.

As you can see in figure 3.14, the health check can be more than an indicator of what's up and what's down. It also can give information about the state of the server on which the microservice instance is running. Spring Actuator lets you change the default configuration via the application properties file. For example:

```
management.endpoints.web.base-path=/
management.endpoints.enabled-by-default=false
management.endpoint.health.enabled=true
management.endpoint.health.show-details=always
management.health.db.enabled=false
management.health.diskspace.enabled=true
```

The first line allows you to set the base path for all Actuator services (for example, the health endpoint is now exposed in the `http://localhost:8080/health` URL). The remaining lines allow you to disable the default services and enable the services you want to use.

NOTE If you are interested in knowing all of the services exposed by Spring Actuator, we recommend you read the following Spring document:

https://docs.spring.io/spring-boot/docs/current/reference/html/production-ready-endpoints.html.

3.5 Pulling the perspectives together

Microservices in the cloud seem deceptively simple. But to be successful with them, you need to have an integrated view that pulls the perspective of the architect, the developer, and the DevOps engineer together into a cohesive vision. The key takeaways for each of these perspectives are as follows:

- *Architect*—Focus on the natural contours of your business problem. Describe your business problem domain and listen to the story you're telling. Target microservice candidates will emerge. Remember, too, that it's better to start with a coarse-grained microservice and refactor back to smaller services than it is to start with a large group of small services. Microservice architectures, like most good architectures, are emergent and not preplanned to the minute.
- *Software engineer (aka developer)*—The fact that the service is small doesn't mean good design principles get thrown out the window. Focus on building a layered service, where each layer in the service has discrete responsibilities. Avoid the temptation to build frameworks in your code and try to make each microservice completely independent. Premature framework design and adoption can have massive maintenance costs later in the lifecycle of the application.
- *DevOps engineer*—Services don't exist in a vacuum. Establish the lifecycle of your services early. The DevOps perspective needs to focus not only on how to automate the building and deployment of a service, but also on how to monitor the health of the service and react when something goes wrong. Operationalizing a service often takes more work and forethought than writing business logic. In appendix C, we'll explain how to achieve this with Prometheus and Grafana.

mmary

- To be successful with microservices, you need to integrate three team perspectives: the architect, the developer, and the DevOps.
- Microservices, while a powerful architectural paradigm, have their benefits and trade-offs. Not all applications should be microservice applications.
- From an architect's perspective, microservices are small, self-contained, and distributed. Microservices should have narrow boundaries and manage a small set of data.
- From a developer's perspective, microservices are typically built using a REST-style design, with JSON as the payload for sending and receiving data from the service.
- The main goal of internationalization is to develop applications that offer content in multiple formats and languages.
- HATEOAS stands for Hypermedia as the Engine of Application State. Spring HATEOAS is a small project that allows us to create APIs that follow the HATEOAS principle of displaying the related links for a given resource.
- From a DevOps perspective, how a microservice is packaged, deployed, and monitored is of critical importance.
- Out of the box, Spring Boot allows you to deliver a service as a single executable JAR file. An embedded Tomcat server in the produced JAR file hosts the service.
- Spring Actuator, which is included with the Spring Boot framework, exposes information about the operational health of the service along with information about the run time of the service.

Previous chapter
< 2 Exploring the microservices world with Spring Cloud

Next chapter
4 Welcome to Docker >